

FM-GPT Tutorial

1 Example Data Generation

We will first generate a synthetic dataset using the included functions. For this example, we will consider a set of 10 continuous outcomes which can be represented by 3 factors. We will generate $n_1 = 100$ samples in the eQTL reference dataset and $n_2 = 1000$ samples in the GWAS dataset. We will simulate a singular region of 10 genes, each gene containing 10 cis-SNPs. The LD matrix for the SNPs will be $\rho_{\text{within}} = 0.66$ and $\rho_{\text{between}} = 0.33$. We will assume approximately 1 gene is causal of the 10 in this region, with a gene expression heritability of 10%. We will also specify the phenotype heritability of approximately 5%.

```
1 set.seed(1423)
2
3 dat <- gen_twas_sets(n_qtl = 100, n_gwas = 5000, n_genes = 10,
  ↳ snps_per_gene = 10, block_sizes = rep(10, 10), rho_between = 0.33,
  ↳ rho_within = 0.66, response_types = c(10, 0, 0), k = 3, maf =
  ↳ 0.24, choice = "genes", sig_rho = 0.66, pi0 = 1, pi1 = 0.1, pi2 =
  ↳ 1, expression_heritability = 0.1, sigma2 = 1, binflate = 0.05)
```

The `dat` object contains the following important elements:

- `G_qtl`, the genotype data for the eQTL study
- `G_gwas`, the genotype data for the GWAS study
- `E_qtl`, the gene expression data for the eQTL study
- `Yl` and `Yo`, the latent factors and observed outcomes
- `Lambda`, the loading matrix
- `beta`, the gene causality pattern for the latent factors
- `responses` the categorization of the outcome type (e.g. continuous, binary, count)

The gene causality pattern for this example looks like this:

```

1  0.0000000 0.0000000 0.0000000
2  0.0000000 0.0000000 0.0000000
3  0.1715735 0.3307978 0.1299771
4  0.0000000 0.0000000 0.0000000
5  0.0000000 0.0000000 0.0000000
6  0.0000000 0.0000000 0.0000000
7  0.0000000 0.0000000 0.0000000
8  0.0000000 0.0000000 0.0000000
9  0.0000000 0.0000000 0.0000000
10 0.0000000 0.0000000 0.0000000

```

So, we have one causal gene in this region (gene # 3).

2 Computation

Now we shall estimate the predicted expression from the synthetic data. We will use a spike-and-slab regression implementation.

```

1  est_coef_mat <- matrix(nrow = nrow(dat$SNP_coef), ncol =
   ↪   ncol(dat$SNP_coef))
2
3  for(j in 1:ncol(dat$SNP_coef)) {
4    tmp_out <- lm.spike(scale(dat$E_qtl[,j], scale = FALSE) ~
   ↪   scale(dat$G_qtl, scale = FALSE)+0, niter = 1000)
5    est_coef_mat[,j] <- apply(tmp_out$beta[751:1000,], 2, median)
6  }

```

For example, for the first gene, here is the weight vector for its 10 cis-SNPs from `est_coef_mat[1:10,1]`:

```

1  0.000000 0.000000 0.000000 0.000000 -1.719266 0.000000 0.000000
2  0.000000 0.000000 -1.101098

```

So, there are two cis-SNPs which contribute to this gene's expression. Now, we will generate the predicted gene expression and fit the model.

```

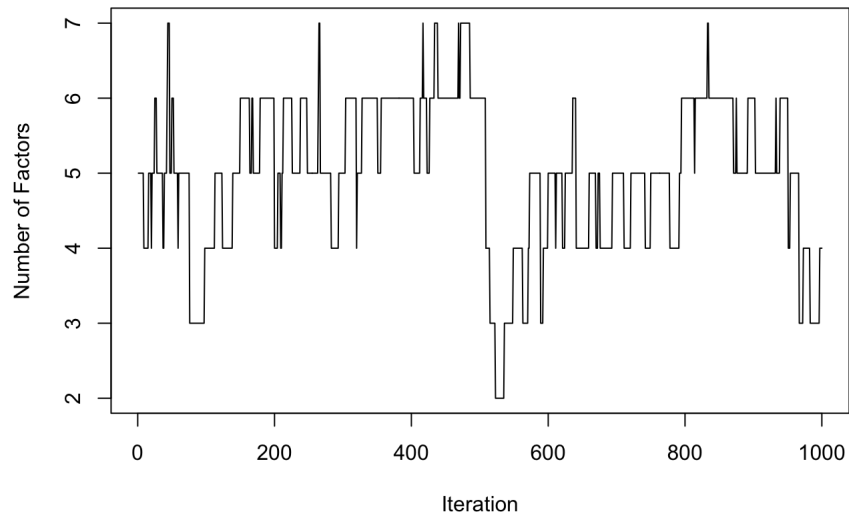
1  E_gwas <- dat$G_gwas %*% est_coef_mat
2
3  fit <- FMGPT(X = scale(E_gwas, scale = FALSE), Y = scale(dat$Yo, scale
   ↪   = FALSE), A = 0*dat$A, group = seq(1, 10), response_types =
   ↪   dat$responses, epsilon = 0.4, B = 3000, burnin = 2000, thin = 1,
   ↪   k_fix = NULL, scale.eta = FALSE)

```

In the model statement above, we center both the outcome and the predicted gene expressions, specify that the adaptive factor algorithm should use $\epsilon = 0.4$ as a loading cutoff, and allow FM-GPT to estimate the number of factors (`k_fix = NULL`). We run the model for a total of 3000 iterations, 2000 of which are burnin.

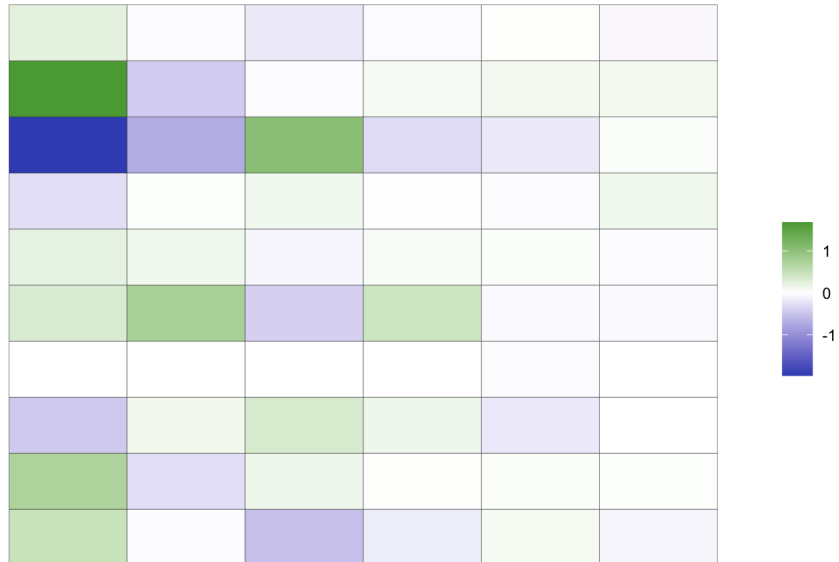
We can first look at the trace plot of the number of factors fit:

```
1 ts.plot(fit$ksstar, xlab = "Iteration", ylab = "Number of Factors")
```



We can also view the matrix loading pattern. Based on the traceplot, we will view the loadings for the first six factors:

```
1 infinitefactor::plotmat(t(apply(fit$lambda, c(2, 3), mean, na.rm =
  ↪ TRUE))[,1:6])
```

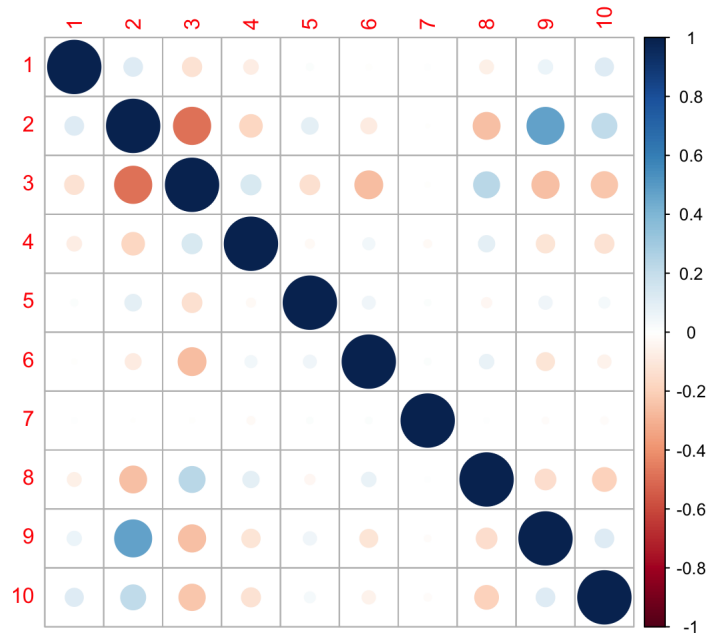


Clearly, the magnitude of the first three factors are much higher than the rest of the factors. We can also look at more classical determinations of number of factors such as a correlation plot and scree plot.

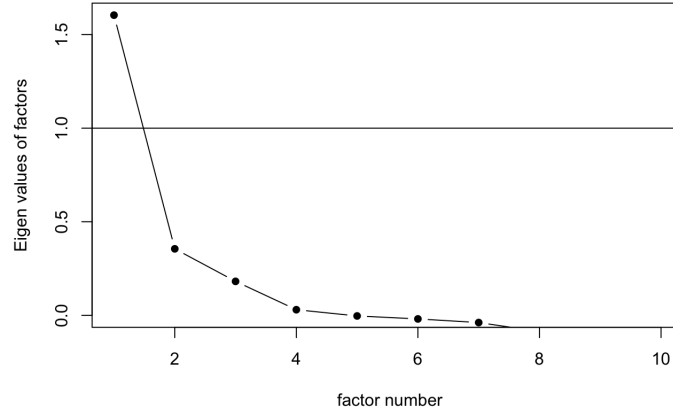
```

1 corrplot::corrplot(cor(dat$Yo))
2 psych::scree(cor(dat$Yo), pc = FALSE)

```



Scree plot



Looking at the trace plot of the number of factors which appears to be decreasing, the fact that factors 4-6 have relatively small magnitude, the visual inspection of the correlation matrix which appears to have three blocks, and the flattening of the scree plot after factor 3, we will assume that FM-GPT has fit 3 factors.

Now, we can look at the posterior mean of the loading matrix from FM-GPT:

```
1 t(apply(fit$lambda, c(2, 3), mean, na.rm = TRUE))[1:3]
```

```

1  0.228136006 -0.021832330 -0.188681266
2  1.678967752 -0.421212221 -0.026450774
3  -1.975485138 -0.720422383  1.050560210
4  -0.275308863  0.027476766  0.112662100
5  0.208116483  0.127676866 -0.068475715
6  0.312933988  0.765620239 -0.387355095
7  -0.003455489  0.009329259  0.002512951
8  -0.447556340  0.106884653  0.345457250
9  0.719770599 -0.270749420  0.131224191
10 0.471726854 -0.027442417 -0.529810597

```

Now, to determine which loadings to suppress to zero, we will apply the post-hoc unit scaling to each iteration of the loading matrix:

```

1  lambda_draws <- fit$lambda[,1:3,]
2  for(b in 1:750) {
3    for(k in 1:3) {
4      lambda_draws[b,k,] <- lambda_draws[b,k,] /
        ↪ sqrt(sum(lambda_draws[b,k,]^2))
5    }
6  }
7  t(apply(lambda_draws, c(2, 3), mean, na.rm = TRUE))[,1:3]

```

```

1  0.125128803 -0.0237267830 -0.108230296
2  0.871217714 -0.3160903453 -0.037762536
3  -1.103193680 -0.5374466405  0.314718766
4  -0.142874434  0.0007746511  0.073747340
5  0.106915864  0.1040648931  0.005468073
6  0.193282812  0.5339733095  0.071825144
7  -0.004489443 -0.0034926657  0.018296691
8  -0.256985029  0.0552749793  0.273067655
9  0.365444884 -0.2025981749  0.021096815
10 0.267596961 -0.0043584861 -0.364186470

```

This matrix gives the relative magnitude of each loading in each factor. We'll keep only those loadings with an absolute magnitude > 0.10.

```

1  lambda_keep <- t(apply(lambda_draws, c(2, 3), mean, na.rm =
        ↪ TRUE))[,1:3]
2  lambda_keep[abs(lambda_keep) < 0.1] <- 0
3  t(apply(fit$lambda, c(2, 3), mean, na.rm = TRUE))[,1:3] *
        ↪ (abs(lambda_keep) > 0.10)

```

```

1  0.2281360  0.0000000 -0.1886813
2  1.6789678 -0.4212122  0.0000000

```

```

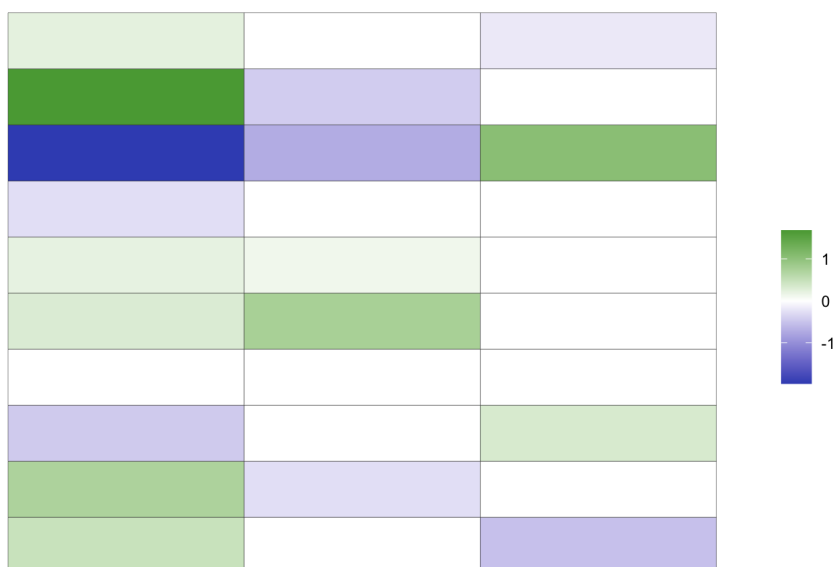
3  -1.9754851 -0.7204224  1.0505602
4  -0.2753089  0.0000000  0.0000000
5   0.2081165  0.1276769  0.0000000
6   0.3129340  0.7656202  0.0000000
7   0.0000000  0.0000000  0.0000000
8  -0.4475563  0.0000000  0.3454572
9   0.7197706 -0.2707494  0.0000000
10  0.4717269  0.0000000 -0.5298106

```

```

1  infinitefactor::plotmat(t(apply(fit$lambda, c(2, 3), mean, na.rm =
   ↪ TRUE))[,1:3] * (abs(lambda_keep) > 0.10))

```

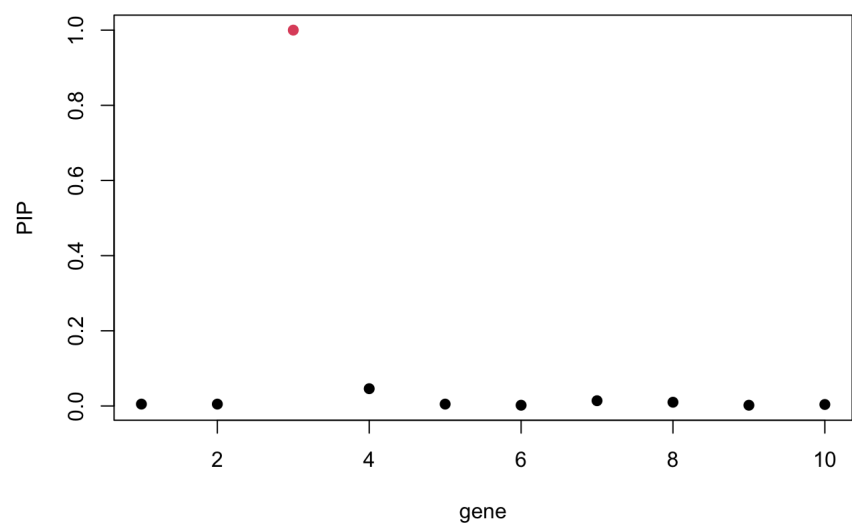


Finally, we can view the PIPs from the model:

```

1  plot(twas_pip(fit, 3), xlab = "gene", ylab = "PIP", col = dat$A+1, pch
   ↪ = 19)

```



Clearly, FM-GPT has prioritized the correct gene (# 3). All other genes in the region have insignifacnt PIPs.